

Title: DSC 680: Project 1: Evaluating MLB Trade Deadline Moves Through Predicted WAR

Author: Matthew Heinen

Date: 17 September 2025

Modified By: Matthew Heinen

Description: This project reads in cleans and gathers the essential data to train and evaluate predictive model predicting a player's future WAR. This project then uses these models to project future player performance of players who were traded at the MLB trade deadline. This is done with the purpose of evaluating the value of return each team receives in the trade.

Read in / Wrangle Data

Here we will read in data from all of our sources as well as merge them into one readable file so that we have all the pitcher statistics as well as the corresponding WAR.

```
In [230... # First read in data from MLB Statcast

# Import Packages
import pandas as pd

# read csv
player_stats = pd.read_csv('stats.csv')

#preview
player_stats.head(5)
```

```
Out[230... last_name, first_name player_id year player_age p_game p_formatted_ip pa ab hit single ... breaking_avg_break breaking_range_speed

0 Pérez, Oliver 424144 2021 39 5 3.2 18 17 5 5 ... 9.7 1.5
1 Wainwright, Adam 425794 2021 39 32 206.1 828 765 168 111 ... 22.2 1.9
2 Greinke, Zack 425844 2021 37 30 171.0 697 652 164 105 ... 12.5 7.2
3 Santana, Ervin 429722 2021 38 38 65.1 277 251 65 38 ... 5.4 2.5
4 Kazmir, Scott 431148 2021 37 5 11.1 55 48 15 9 ... 8.4 2.0
```

5 rows × 207 columns

```
In [231... # find the shape of the dataframe to determine how many variables and records in the set
player_stats.shape
```

```
Out[231... (3929, 207)
```

```
In [232... # Split the "last_name, first_name" column and create a new "name" column
player_stats['name'] = player_stats['last_name, first_name'].str.split(', ').apply(
    lambda x: f"{x[1]} {x[0]}" if len(x) > 1 else x[0]
)

# Preview the dataframe with the new column
player_stats.head(5)
```

```
Out[232... last_name, first_name player_id year player_age p_game p_formatted_ip pa ab hit single ... breaking_range_speed n_offspeed_formatted

0 Pérez, Oliver 424144 2021 39 5 3.2 18 17 5 5 ... 1.5 NaN
1 Wainwright, Adam 425794 2021 39 32 206.1 828 765 168 111 ... 1.9 6.4
2 Greinke, Zack 425844 2021 37 30 171.0 697 652 164 105 ... 7.2 22.3
3 Santana, Ervin 429722 2021 38 38 65.1 277 251 65 38 ... 2.5 3.3
4 Kazmir, Scott 431148 2021 37 5 11.1 55 48 15 9 ... 2.0 30.9
```

5 rows × 208 columns

```
In [233... # Read in WAR data from fangraphs.
# YEar 2024

#read csv
pit_war_24 = pd.read_csv('fangraphs-war-leaderboard.csv')

# Add a 'year' column with value 2024 to all records
pit_war_24['year'] = 2024

# Preview df
pit_war_24.head(5)
```

```
Out[233...
      Name Team   PA   IP  Bat WAR  Pit WAR  Total WAR  PlayerId  year
0  Aaron Judge  NYY  704.0  NaN  11.329109    NaN  11.329109    15640  2024
1  Bobby Witt Jr.  KCR  709.0  NaN  10.479079    NaN  10.479079    25764  2024
2  Shohei Ohtani  LAD  731.0  NaN  8.952518    NaN  8.952518    19755  2024
3   Juan Soto  NYY  713.0  NaN  8.285021    NaN  8.285021    20123  2024
4  Gunnar Henderson  BAL  719.0  NaN  7.940532    NaN  7.940532    26289  2024
```

```
In [234... # Read in WAR data from fangraphs.
# YEar 2025

#read csv
pit_war_25 = pd.read_csv('fangraphs-war-leaderboard (1).csv')

# Add a 'year' column with value 2025 to all records
pit_war_25['year'] = 2025

# Preview df
pit_war_25.head(5)
```

```
Out[234...
      Name Team   PA   IP  Bat WAR  Pit WAR  Total WAR  PlayerId  year
0  Aaron Judge  NYY  652.0  NaN  9.200006    NaN  9.200006    15640  2025
1  Shohei Ohtani  LAD  704.0  41.0  7.250475  1.579565  8.830040    19755  2025
2   Cal Raleigh  SEA  678.0  NaN  8.781865    NaN  8.781865    21534  2025
3  Bobby Witt Jr.  KCR  659.0  NaN  7.704574    NaN  7.704574    25764  2025
4  Geraldo Perdomo  ARI  692.0  NaN  6.908167    NaN  6.908167    22799  2025
```

```
In [235... # Read in WAR data from fangraphs.
# YEar 2023

#read csv
pit_war_23 = pd.read_csv('fangraphs-war-leaderboard (2).csv')

# Add a 'year' column with value 2023 to all records
pit_war_23['year'] = 2023

# Preview df
pit_war_23.head(5)
```

```
Out[235...
      Name Team   PA   IP  Bat WAR  Pit WAR  Total WAR  PlayerId  year
0  Ronald Acuña Jr.  ATL  735.0  NaN  9.143379    NaN  9.143379    18401  2023
1  Shohei Ohtani  LAA  599.0  132.0  6.581863  2.290122  8.871985    19755  2023
2  Freddie Freeman  LAD  730.0  NaN  7.705377    NaN  7.705377    5361  2023
3  Mookie Betts  LAD  693.0  NaN  7.604586    NaN  7.604586    13611  2023
4   Matt Olson  ATL  720.0  NaN  6.595669    NaN  6.595669    14344  2023
```

```
In [236... # Read in WAR data from fangraphs.
# YEar 2022

#read csv
pit_war_22 = pd.read_csv('fangraphs-war-leaderboard (3).csv')

# Add a 'year' column with value 2022 to all records
pit_war_22['year'] = 2022

# Preview df
pit_war_22.head(5)
```

Out [236...

	Name	Team	PA	IP	Bat WAR	Pit WAR	Total WAR	PlayerId	year
0	Aaron Judge	NY Yankees	696.0	NaN	11.097879	NaN	11.097879	15640	2022
1	Shohei Ohtani	LAA	666.0	166.0	3.614345	5.593072	9.207417	19755	2022
2	Nolan Arenado	STL	620.0	NaN	7.208796	NaN	7.208796	9777	2022
3	Manny Machado	SDP	644.0	NaN	7.128688	NaN	7.128688	11493	2022
4	Jose Altuve	HOU	604.0	NaN	6.861330	NaN	6.861330	5417	2022

In [237...

```
# Read in WAR data from fangraphs.
# YEar 2021

#read csv
pit_war_21 = pd.read_csv('fangraphs-war-leaderboard (4).csv')

# Add a 'year' column with value 2023 to all records
pit_war_21['year'] = 2021

# Preview df
pit_war_21.head(5)
```

Out [237...

	Name	Team	PA	IP	Bat WAR	Pit WAR	Total WAR	PlayerId	year
0	Shohei Ohtani	LAA	639.0	130.3333	5.017931	2.959517	7.977449	19755	2021
1	Corbin Burnes	MIL	59.0	167.0000	0.117416	7.472195	7.589611	19361	2021
2	Zack Wheeler	PHI	73.0	213.3333	0.088868	7.221505	7.310373	10310	2021
3	Trea Turner	- - -	646.0	NaN	7.127414	NaN	7.127414	16252	2021
4	Juan Soto	WSN	654.0	NaN	6.789302	NaN	6.789302	20123	2021

In [238...

```
# Read in WAR data from fangraphs.
# YEar 2020

#read csv
pit_war_20 = pd.read_csv('fangraphs-war-leaderboard (5).csv')

# Add a 'year' column with value 2020 to all records
pit_war_20['year'] = 2020

# Preview df
pit_war_20.head(5)
```

Out [238...

	Name	Team	PA	IP	Bat WAR	Pit WAR	Total WAR	PlayerId	year
0	Fernando Tatis Jr.	SDP	257.0	NaN	3.314397	NaN	3.314397	19709	2020
1	Shane Bieber	CLE	NaN	77.3333	0.000000	3.149900	3.149900	19427	2020
2	José Ramírez	CLE	254.0	NaN	3.109814	NaN	3.109814	13510	2020
3	Freddie Freeman	ATL	262.0	NaN	2.976279	NaN	2.976279	5361	2020
4	Yu Darvish	CHC	NaN	76.0000	0.000000	2.931952	2.931952	13074	2020

In [239...

```
# Append all years of WAR data into a single dataframe
import pandas as pd
pit_war = pd.concat([pit_war_20, pit_war_21, pit_war_22, pit_war_23, pit_war_24, pit_war_25], ignore_index=True)
```

In [240...

```
# perform a series of merges with all of the war dataframes to append the WAR statistics for each player and season.
# Left merge
# on name and year

merged_df = pd.merge(player_stats, pit_war, how='left', left_on=['name', 'year'], right_on=['Name', 'year'])
merged_df.shape
```

Out [240... (3941, 216)

In [241...

```
# Filter to only include records where Total war is not null.

filtered_df = merged_df[merged_df['Total WAR'].notnull()]

# find the shape
filtered_df.shape
```

Out [241... (3893, 216)

Handle missing Data and Outliers.

```
In [242... # Print the columns that are missing values and the percentage of missing values for each column
# Do this for all columns in the dataframe
missing_values = filtered_df.isnull().sum()
missing_percentage = (missing_values / len(filtered_df)) * 100
missing_data = pd.DataFrame({'Missing Values': missing_values, 'Percentage': missing_percentage})
print(missing_data[missing_data['Missing Values'] > 0])
```

	Missing Values	Percentage
p_opp_batting_avg	3893	100.000000
p_opp_on_base_avg	3893	100.000000
avg_swing_speed	1704	43.770871
fast_swing_rate	1704	43.770871
blasts_contact	1704	43.770871
...	...	...
offspeed_avg_break	686	17.621372
offspeed_range_speed	773	19.856152
PA	3462	88.928847
IP	7	0.179810
Pit WAR	7	0.179810

[138 rows x 2 columns]

```
In [243... # Remove the two columns that are missing all values
# 100% missing rate
filtered_df = filtered_df.drop(columns=['p_opp_batting_avg', 'p_opp_on_base_avg'])
```

```
In [244... # Fill the missing values on all othe columns with the median of each column
for column in filtered_df.columns:
    if filtered_df[column].isnull().any():
        median_value = filtered_df[column].median()
        filtered_df[column].fillna(median_value, inplace=True)
```

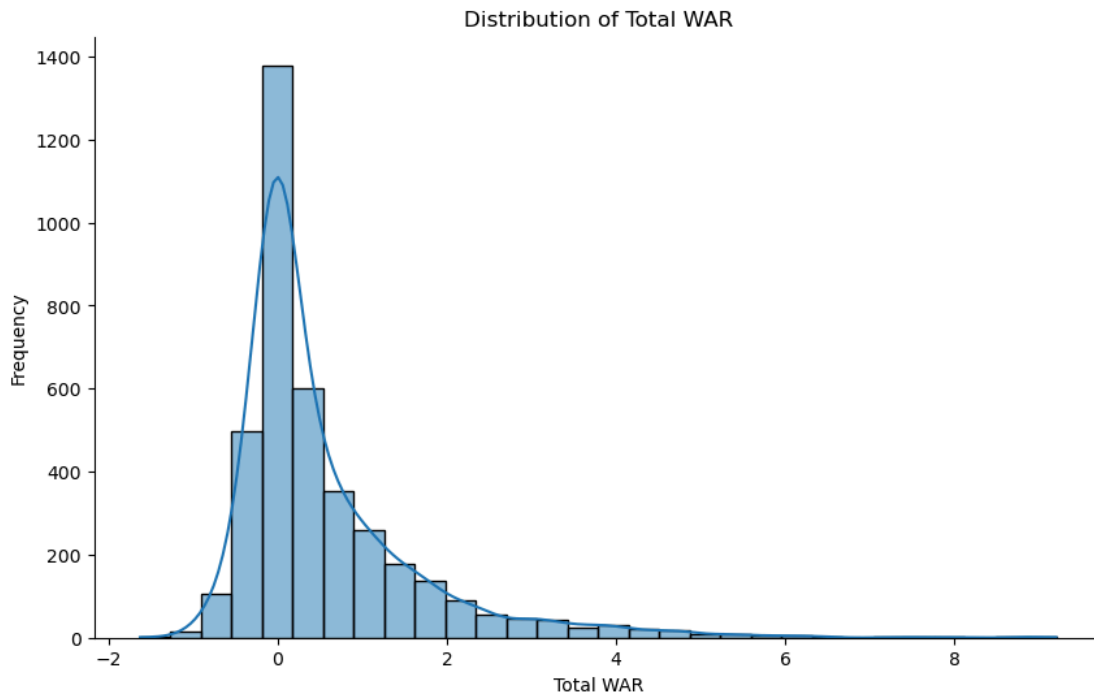
```
In [245... # PReview the filtered Dataframe shape and remaining columns
print(filtered_df.shape)
print(filtered_df.columns)

(3893, 214)
Index(['last_name', 'first_name', 'player_id', 'year', 'player_age', 'p_game',
      'p_formatted_ip', 'pa', 'ab', 'hit', 'single',
      ...,
      'offspeed_range_speed', 'name', 'Name', 'Team', 'PA', 'IP', 'Bat WAR',
      'Pit WAR', 'Total WAR', 'PlayerId'],
      dtype='object', length=214)
```

```
In [246... # Remove personally identifiable information
# Keep one name column
filtered_df = filtered_df.drop(columns=['PlayerId', 'Name', 'last_name', 'first_name', 'Team', 'player_id'])
```

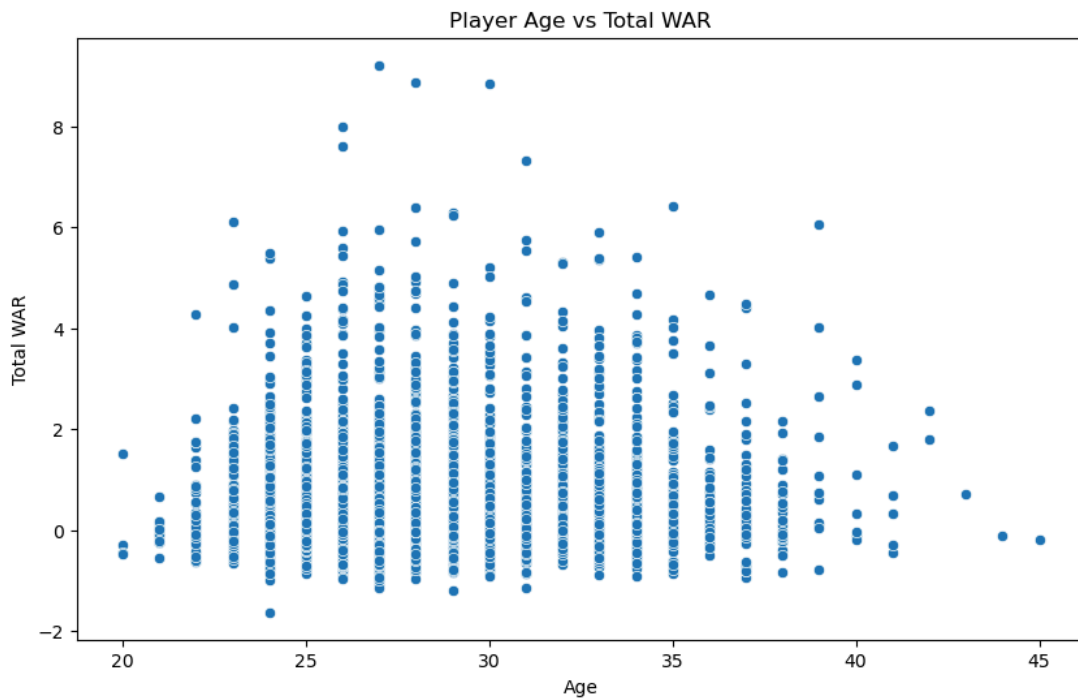
EDA

```
In [247... # Create a histogram of the Total WAR distribution
import matplotlib.pyplot as plt
import seaborn as sns
# EDA
plt.figure(figsize=(10, 6))
sns.histplot(filtered_df['Total WAR'], bins=30, kde=True)
plt.title('Distribution of Total WAR')
plt.xlabel('Total WAR')
plt.ylabel('Frequency')
# Remove top and right borders
sns.despine()
plt.show()
```



As we can see, the Total WAR is skewed to the right as expected. This makes sense because a WAR or win above replacement is a measure of a major league player's value in wins greater than that of a minor league player or a replacement. Players who produce a WAR of zero are seen as replaceable.

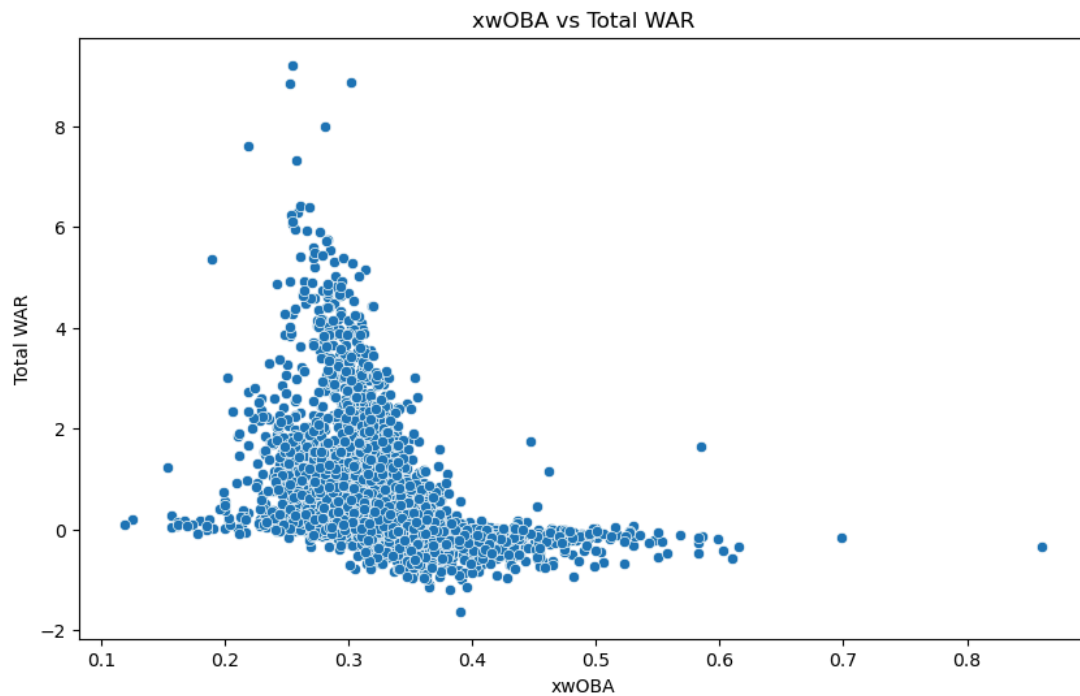
```
In [248... # Build a scatter plot showing relationship between player age and Total WAR
plt.figure(figsize=(10, 6))
sns.scatterplot(x='player_age', y='Total WAR', data=filtered_df)
plt.title('Player Age vs Total WAR')
plt.xlabel('Age')
plt.ylabel('Total WAR')
plt.show()
```



No obvious interaction/relationship between age and war here may be a slight quadratic relationship.

```
In [249... # Show relationship between XwOBA and Total WAR
plt.figure(figsize=(10, 6))
sns.scatterplot(x='xwoba', y='Total WAR', data=filtered_df)
plt.title('xwOBA vs Total WAR')
plt.xlabel('xwOBA')
```

```
plt.ylabel('Total WAR')
plt.show()
```



Generally we see pitchers with low weighted on base average perform better in war.

```
In [281... # Preview the filtered DF
filtered_df.head(5)
```

```
Out[281...
   year  player_age  p_game  p_formatted_ip  pa  ab  hit  single  double  triple  ...  offspeed_avg_break_z_induced  offspeed_avg_break
0  2021         39        5           3.2   18   17    5      5      0      0  ...                5.7                15.9
1  2021         39       32          206.1  828  765  168    111     34      2  ...                7.7                16.0
2  2021         37       30          171.0  697  652  164    105     29      0  ...                4.5                13.7
3  2021         38       38           65.1  277  251   65     38     16      2  ...               12.9                15.4
4  2021         37        5           11.1   55   48   15      9      3      0  ...                7.9                15.7
```

5 rows x 210 columns

Peapare a model data frame to predict Next Season Total War

```
In [251... # Add the column next season total war
# Loop though the years 2020-2024 and add the next season total war to a new column per player
filtered_df['next_season_total_war'] = None
for year in range(2020, 2025):
    next_year = year + 1
    for index, row in filtered_df[filtered_df['year'] == year].iterrows():
        player_name = row['name']
        next_season_war = filtered_df[(filtered_df['name'] == player_name) & (filtered_df['year'] == next_year)]['Total WAR']
        if not next_season_war.empty:
            filtered_df.at[index, 'next_season_total_war'] = next_season_war.values[0]
```

```
In [252... # Re assign to model_df
model_df = filtered_df.copy()
# Perview
filtered_df.head(5)
```

Out[252...

	year	player_age	p_game	p_formatted_ip	pa	ab	hit	single	double	triple	...	offspeed_avg_break_z_induced	offspeed_avg_break
0	2021	39	5	3.2	18	17	5	5	0	0	...	5.7	15.9
1	2021	39	32	206.1	828	765	168	111	34	2	...	7.7	16.0
2	2021	37	30	171.0	697	652	164	105	29	0	...	4.5	13.7
3	2021	38	38	65.1	277	251	65	38	16	2	...	12.9	15.4
4	2021	37	5	11.1	55	48	15	9	3	0	...	7.9	15.7

5 rows × 210 columns

In [253...

```
# Drop rows with null next season total war
# We cannot use these rows to train the model.
model_df = model_df[model_df['next_season_total_war'].notnull()]
```

In [254...

```
#Find the unique position values and their counts.
model_df.shape
```

Out[254...

(2260, 210)

Train Models

Backwards stepwise regression

In [255...

```
# Create a train test split.
# Import packages.
import pandas as pd
from sklearn.model_selection import train_test_split
# create features
# Drop target variables and unnecessary columns
features = model_df.drop(columns=['name', 'Total WAR', 'next_season_total_war'])
target = model_df['next_season_total_war']
# Split into 80 % training 20% test # use train_test_split
features_train, features_test, target_train, target_test = train_test_split(features,
target, test_size=0.2, random_state=42)

# Ensure all features are numeric for modeling
features_train = features_train.apply(pd.to_numeric, errors='coerce')
features_test = features_test.apply(pd.to_numeric, errors='coerce')
target_train = pd.to_numeric(target_train, errors='coerce')
target_test = pd.to_numeric(target_test, errors='coerce')
```

In [256...

```
# Build stepwise regression with a p value threshold of 0.05

# Load Libraries
import pandas as pd
import statsmodels.api as sm
import warnings
import numpy as np

# Suppress Warning
warnings.simplefilter("ignore", category=RuntimeWarning)

# Add a constant to the model (intercept)
features_train = sm.add_constant(features_train)

# Fit the initial model
model = sm.OLS(target_train, features_train).fit()

# Backward stepwise regression
while True:
    # Get the p-values of the model
    p_values = model.pvalues
    # Find the maximum p-value
    max_p_value = p_values.max()

    # If the maximum p-value is greater than 0.05, remove that variable
    if max_p_value > 0.05:
        # Identify the variable to remove
        variable_to_remove = p_values.idxmax()
        # Skip removing 'const'
        if variable_to_remove == 'const':
```

```
        break
        # Remove the variable from the model
        features_train = features_train.drop(variable_to_remove, axis=1)
        # Remove from test data as well
        features_test = features_test.drop(variable_to_remove, axis=1)
        # Refit the model
        model = sm.OLS(target_train, features_train).fit()
    else:
        break

# Print the summary of the final model # Commented out for length, Includes too many features
print("Final model summary:")
print(model.summary())

# Prepare the test set for prediction
# Add constant to test set
features_test_final = features_test.copy()
#features_test_final = sm.add_constant(features_test)
```


Final model summary:

OLS Regression Results

Dep. Variable:	next_season_total_war	R-squared:	0.497
Model:	OLS	Adj. R-squared:	0.461
Method:	Least Squares	F-statistic:	13.88
Date:	Sun, 05 Oct 2025	Prob (F-statistic):	8.22e-177
Time:	14:57:08	Log-Likelihood:	-2247.4
No. Observations:	1808	AIC:	4737.
Df Residuals:	1687	BIC:	5402.
Df Model:	120		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	-84.6367	257.946	-0.328	0.743	-590.564	421.291
year	-0.0237	0.023	-1.039	0.299	-0.068	0.021
player_age	-0.0091	0.007	-1.313	0.189	-0.023	0.004
p_game	-0.0069	0.002	-3.871	0.000	-0.010	-0.003
p_formatted_ip	-0.0199	0.019	-1.072	0.284	-0.056	0.017
pa	0.0241	0.008	2.846	0.004	0.008	0.041
ab	-0.0223	0.008	-2.626	0.009	-0.039	-0.006
single	0.0142	0.007	2.087	0.037	0.001	0.028
double	-0.0133	0.009	-1.467	0.143	-0.031	0.004
triple	0.0150	0.022	0.693	0.488	-0.028	0.058
strikeout	0.0174	0.003	5.662	0.000	0.011	0.023
walk	-0.0247	0.009	-2.720	0.007	-0.043	-0.007
on_base_plus_slg	-13.0223	14.556	-0.895	0.371	-41.571	15.527
babip	3.8082	3.013	1.264	0.206	-2.100	9.717
p_total_stolen_base	0.0054	0.006	0.874	0.382	-0.007	0.018
p_lob	-0.0079	0.003	-2.372	0.018	-0.014	-0.001
xba	15.5728	15.079	1.033	0.302	-14.002	45.148
xwoba	-5.4252	5.310	-1.022	0.307	-15.839	4.989
xobp	16.5819	15.098	1.098	0.272	-13.030	46.194
xiso	12.8352	14.547	0.882	0.378	-15.697	41.367
wobacon	-10.3442	7.972	-1.298	0.195	-25.980	5.291
xwobacon	15.0930	8.070	1.870	0.062	-0.735	30.921
bacon	9.6890	10.277	0.943	0.346	-10.468	29.846
xbacon	-18.2628	9.292	-1.966	0.050	-36.487	-0.038
xbadiff	-7.8854	9.019	-0.874	0.382	-25.574	9.804
xslgdiff	15.6371	12.423	1.259	0.208	-8.729	40.003
wobadiff	12.2463	11.805	1.037	0.300	-10.908	35.401
squared_up_contact	0.0123	0.014	0.847	0.397	-0.016	0.041
squared_up_swing	-0.0182	0.017	-1.079	0.281	-0.051	0.015
avg_swing_length	-0.1487	0.188	-0.791	0.429	-0.518	0.220
swords	-0.0088	0.006	-1.490	0.136	-0.020	0.003
sweet_spot_percent	-0.0092	0.007	-1.378	0.169	-0.022	0.004
barrel_batted_rate	0.0896	0.034	2.652	0.008	0.023	0.156
solidcontact_percent	0.0731	0.033	2.244	0.025	0.009	0.137
flareburner_percent	0.0620	0.032	1.964	0.050	8.28e-05	0.124
poorlyunder_percent	0.0610	0.032	1.900	0.058	-0.002	0.124
poorlytopped_percent	0.0602	0.031	1.919	0.055	-0.001	0.122
poorlyweak_percent	0.0564	0.032	1.753	0.080	-0.007	0.119
avg_best_speed	-0.0112	0.014	-0.802	0.423	-0.038	0.016
avg_hyper_speed	-0.0329	0.043	-0.765	0.444	-0.117	0.051
edge_percent	0.0094	0.008	1.183	0.237	-0.006	0.025
f_strike_percent	0.0091	0.004	2.269	0.023	0.001	0.017
groundballs_percent	0.2591	0.222	1.168	0.243	-0.176	0.694
flyballs_percent	0.2560	0.222	1.153	0.249	-0.179	0.691
linedrives_percent	0.2554	0.222	1.151	0.250	-0.180	0.691
popups_percent	0.2514	0.222	1.132	0.258	-0.184	0.687
n_ff_formatted	-0.0035	0.002	-1.526	0.127	-0.008	0.001
ff_avg_speed	0.0481	0.021	2.334	0.020	0.008	0.088
ff_avg_break_z	-0.0182	0.012	-1.546	0.122	-0.041	0.005
sl_avg_speed	-0.1608	0.115	-1.392	0.164	-0.387	0.066
sl_avg_spin	-0.0002	0.000	-0.801	0.423	-0.001	0.000
sl_avg_break_x	-0.0064	0.006	-1.149	0.251	-0.017	0.005
sl_avg_break_z	0.1734	0.116	1.494	0.135	-0.054	0.401
sl_avg_break_z_induced	-0.1792	0.116	-1.548	0.122	-0.406	0.048
n_ch_formatted	-0.0259	0.014	-1.789	0.074	-0.054	0.003
ch_avg_speed	-0.6009	0.218	-2.751	0.006	-1.029	-0.173
ch_avg_break_x	-0.0246	0.017	-1.473	0.141	-0.057	0.008
ch_avg_break_z	0.7016	0.255	2.749	0.006	0.201	1.202
ch_avg_break_z_induced	-0.7176	0.251	-2.856	0.004	-1.210	-0.225
ch_avg_break	0.0466	0.030	1.576	0.115	-0.011	0.105
n_cu_formatted	-0.0024	0.003	-0.714	0.476	-0.009	0.004
cu_avg_speed	-0.1190	0.076	-1.558	0.119	-0.269	0.031
cu_avg_break_x	-0.0056	0.004	-1.273	0.203	-0.014	0.003
cu_avg_break_z	0.1178	0.067	1.758	0.079	-0.014	0.249
cu_avg_break_z_induced	-0.1263	0.066	-1.906	0.057	-0.256	0.004
n_si_formatted	-0.0029	0.003	-1.125	0.261	-0.008	0.002
si_avg_speed	0.2258	0.108	2.091	0.037	0.014	0.438
si_avg_break_x	-0.0043	0.004	-1.157	0.247	-0.012	0.003
si_avg_break_z	-0.3357	0.155	-2.162	0.031	-0.640	-0.031
si_avg_break_z_induced	0.3344	0.157	2.128	0.033	0.026	0.643
si_avg_break	-0.0295	0.020	-1.483	0.138	-0.068	0.009

n_fc_formatted	-0.0081	0.003	-2.758	0.006	-0.014	-0.002
fc_avg_speed	0.0289	0.014	2.004	0.045	0.001	0.057
fc_range_speed	-0.1456	0.145	-1.007	0.314	-0.429	0.138
n_fs_formatted	-0.0234	0.014	-1.721	0.085	-0.050	0.003
fs_avg_speed	-0.2001	0.184	-1.085	0.278	-0.562	0.162
fs_avg_spin	0.0003	0.000	1.315	0.189	-0.000	0.001
fs_avg_break_x	-0.0267	0.014	-1.843	0.065	-0.055	0.002
fs_avg_break_z	0.1944	0.211	0.921	0.357	-0.220	0.609
fs_avg_break_z_induced	-0.1875	0.212	-0.885	0.376	-0.603	0.228
n_kn_formatted	-0.5906	0.312	-1.895	0.058	-1.202	0.021
kn_avg_speed	3.1048	1.810	1.715	0.086	-0.446	6.655
kn_avg_spin	0.0070	0.006	1.162	0.245	-0.005	0.019
kn_avg_break_z	-2.5727	1.409	-1.826	0.068	-5.336	0.191
kn_avg_break_z_induced	2.1055	1.390	1.515	0.130	-0.621	4.832
kn_avg_break	1.1296	0.628	1.799	0.072	-0.102	2.361
kn_range_speed	11.6489	8.379	1.390	0.165	-4.785	28.082
st_avg_speed	-0.1824	0.117	-1.556	0.120	-0.412	0.048
st_avg_spin	-0.0006	0.000	-2.499	0.013	-0.001	-0.000
st_avg_break_x	-0.0027	0.004	-0.751	0.453	-0.010	0.004
st_avg_break_z	0.2283	0.109	2.101	0.036	0.015	0.441
st_avg_break_z_induced	-0.2353	0.109	-2.167	0.030	-0.448	-0.022
st_avg_break	0.0328	0.018	1.861	0.063	-0.002	0.067
st_range_speed	-0.2468	0.179	-1.382	0.167	-0.597	0.104
sv_avg_spin	-0.0008	0.000	-1.713	0.087	-0.002	0.000
sv_avg_break	0.0425	0.040	1.052	0.293	-0.037	0.122
sv_range_speed	-0.5149	0.678	-0.759	0.448	-1.845	0.816
n_fo_formatted	0.1763	0.145	1.218	0.223	-0.108	0.460
fo_avg_speed	-2.8002	1.290	-2.171	0.030	-5.330	-0.270
fo_avg_spin	0.0166	0.008	2.066	0.039	0.001	0.032
fo_avg_break_x	2.5024	1.567	1.597	0.110	-0.571	5.576
n_sc_formatted	-3.4298	10.229	-0.335	0.737	-23.493	16.634
sc_avg_break_z	0.5537	1.311	0.422	0.673	-2.017	3.125
sc_range_speed	-1.7149	5.115	-0.335	0.737	-11.747	8.317
n_fastball_formatted	0.0070	0.003	2.457	0.014	0.001	0.013
fastball_avg_spin	0.0003	0.000	1.482	0.139	-9.3e-05	0.001
fastball_avg_break_x	0.0089	0.005	1.724	0.085	-0.001	0.019
fastball_avg_break	0.0262	0.016	1.644	0.100	-0.005	0.057
breaking_avg_speed	0.2050	0.107	1.909	0.056	-0.006	0.416
breaking_avg_spin	0.0002	0.000	1.019	0.308	-0.000	0.001
breaking_avg_break_z	-0.2192	0.102	-2.158	0.031	-0.418	-0.020
breaking_avg_break_z_induced	0.2079	0.102	2.041	0.041	0.008	0.408
breaking_range_speed	0.0734	0.028	2.653	0.008	0.019	0.128
n_offspeed_formatted	0.0322	0.014	2.224	0.026	0.004	0.061
offspeed_avg_speed	0.5699	0.209	2.725	0.007	0.160	0.980
offspeed_avg_break_x	0.0235	0.017	1.374	0.170	-0.010	0.057
offspeed_avg_break_z	-0.6560	0.245	-2.678	0.007	-1.137	-0.176
offspeed_avg_break_z_induced	0.6631	0.241	2.749	0.006	0.190	1.136
offspeed_avg_break	-0.0303	0.028	-1.090	0.276	-0.085	0.024
offspeed_range_speed	0.2958	0.099	2.972	0.003	0.101	0.491
PA	0.0041	0.002	2.060	0.040	0.000	0.008
Bat WAR	0.5157	0.266	1.938	0.053	-0.006	1.038
Pit WAR	0.2570	0.064	4.034	0.000	0.132	0.382
=====						
Omnibus:	270.603	Durbin-Watson:	2.004			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	814.177			
Skew:	0.765	Prob(JB):	1.60e-177			
Kurtosis:	5.909	Cond. No.	1.00e+16			
=====						

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [2] The smallest eigenvalue is 6.77e-22. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

```
In [257... # Evaluate Fit

# Load Libraries
import numpy as np
from sklearn.metrics import r2_score, mean_squared_error
# Prepare the test set for prediction
features_test_final = features_test.copy()

# Add 'const' if needed
if 'const' in features_train.columns and 'const' not in features_test_final.columns:
    features_test_final = sm.add_constant(features_test_final, has_constant='add')

# Ensure columns match
features_test_final = features_test_final[features_train.columns]
# Predictions
train_predictions = model.predict(features_train)
test_predictions = model.predict(features_test_final)

# R² Calculation
r2_train = r2_score(target_train, train_predictions)
r2_test = r2_score(target_test, test_predictions)
```

```
# RMSE Calculation
rmse_train = np.sqrt(mean_squared_error(target_train, train_predictions))
rmse_test = np.sqrt(mean_squared_error(target_test, test_predictions))

# Output the results
print(" P-Value Threshold 0.05 Backwards Stepwise Regression")
print(f"Training R²: {r2_train}, Training RMSE: {rmse_train}")
print(f"Test R²: {r2_test}, Test RMSE: {rmse_test}")
```

P-Value Threshold 0.05 Backwards Stepwise Regression
 Training R²: 0.49679595965539214, Training RMSE: 0.8386895772856245
 Test R²: 0.3528225744506932, Test RMSE: 0.9423213644827271

Backwards stepwise regression with PCA and standard scaler

```
In [258... # Create a train test split.
# Import packages.
import pandas as pd
from sklearn.model_selection import train_test_split
# create features
# Drop target variables and unnecessary columns
features = model_df.drop(columns=['name', 'Total WAR', 'next_season_total_war'])
target = model_df['next_season_total_war']
# Split into 80 % training 20% test # use train_test_split
features_train, features_test, target_train, target_test = train_test_split(features,
target, test_size=0.2, random_state=42)

# Ensure all features are numeric for modeling
features_train = features_train.apply(pd.to_numeric, errors='coerce')
features_test = features_test.apply(pd.to_numeric, errors='coerce')
target_train = pd.to_numeric(target_train, errors='coerce')
target_test = pd.to_numeric(target_test, errors='coerce')
```

```
In [259... import pandas as pd
import statsmodels.api as sm
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import warnings
import numpy as np

warnings.simplefilter("ignore", category=RuntimeWarning)

scaler = StandardScaler()
features_train_scaled = scaler.fit_transform(features_train)
features_test_scaled = scaler.transform(features_test)

pca = PCA()
features_train_pca = pca.fit_transform(features_train_scaled)
features_test_pca = pca.transform(features_test_scaled)

# Convert to DataFrame for .drop() support
features_train_pca = pd.DataFrame(features_train_pca)
features_test_pca = pd.DataFrame(features_test_pca)

# Add constant for intercept
features_train_pca = sm.add_constant(features_train_pca)
features_test_pca = sm.add_constant(features_test_pca)

# Reset indices to ensure alignment
features_train_pca = features_train_pca.reset_index(drop=True)
features_test_pca = features_test_pca.reset_index(drop=True)
target_train = target_train.reset_index(drop=True)
target_test = target_test.reset_index(drop=True)

# Fit the initial model
model = sm.OLS(target_train, features_train_pca).fit()

while True:
    p_values = model.pvalues
    max_p_value = p_values.max()
    if max_p_value > 0.05:
        variable_to_remove = p_values.idxmax()
        if variable_to_remove == 'const':
            break
        features_train_pca = features_train_pca.drop(variable_to_remove, axis=1)
        features_test_pca = features_test_pca.drop(variable_to_remove, axis=1)
        model = sm.OLS(target_train, features_train_pca).fit()
    else:
        break

# Prepare the test set for prediction
features_test_final = features_test_pca.copy()
```

```
# Print the summary of the final model # Commented out for length  
print("Final model summary:")  
print(model.summary())
```

Final model summary:

OLS Regression Results

```

=====
Dep. Variable:    next_season_total_war    R-squared:                0.463
Model:            OLS                     Adj. R-squared:           0.445
Method:           Least Squares            F-statistic:              26.02
Date:             Sun, 05 Oct 2025          Prob (F-statistic):       3.06e-193
Time:             14:57:14                 Log-Likelihood:           -2305.9
No. Observations: 1808                    AIC:                     4730.
Df Residuals:     1749                    BIC:                     5054.
Df Model:         58
Covariance Type:  nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	0.6941	0.021	33.509	0.000	0.653	0.735
0	-0.0989	0.005	-20.086	0.000	-0.109	-0.089
1	0.0969	0.005	17.818	0.000	0.086	0.108
2	0.0506	0.006	8.647	0.000	0.039	0.062
3	0.0432	0.007	6.033	0.000	0.029	0.057
4	0.0220	0.007	3.014	0.003	0.008	0.036
5	-0.0384	0.008	-4.644	0.000	-0.055	-0.022
6	0.0294	0.009	3.406	0.001	0.012	0.046
7	-0.0494	0.009	-5.607	0.000	-0.067	-0.032
10	-0.0277	0.010	-2.700	0.007	-0.048	-0.008
11	-0.0335	0.010	-3.226	0.001	-0.054	-0.013
13	-0.0539	0.011	-5.035	0.000	-0.075	-0.033
16	0.0365	0.012	3.101	0.002	0.013	0.060
17	0.0440	0.012	3.682	0.000	0.021	0.067
18	-0.0548	0.012	-4.483	0.000	-0.079	-0.031
19	0.0450	0.013	3.566	0.000	0.020	0.070
20	0.0312	0.013	2.453	0.014	0.006	0.056
24	-0.0418	0.014	-3.044	0.002	-0.069	-0.015
25	0.0406	0.014	2.850	0.004	0.013	0.068
26	0.1207	0.014	8.379	0.000	0.092	0.149
28	0.0520	0.015	3.426	0.001	0.022	0.082
29	-0.0801	0.016	-5.094	0.000	-0.111	-0.049
31	0.0369	0.016	2.333	0.020	0.006	0.068
33	0.0341	0.017	2.059	0.040	0.002	0.066
34	-0.0538	0.017	-3.202	0.001	-0.087	-0.021
36	0.0880	0.018	5.020	0.000	0.054	0.122
37	0.0453	0.018	2.560	0.011	0.011	0.080
38	0.0396	0.018	2.192	0.029	0.004	0.075
43	0.0476	0.019	2.486	0.013	0.010	0.085
47	-0.0431	0.020	-2.160	0.031	-0.082	-0.004
51	-0.1550	0.021	-7.483	0.000	-0.196	-0.114
53	-0.0574	0.021	-2.703	0.007	-0.099	-0.016
54	0.0509	0.021	2.375	0.018	0.009	0.093
56	-0.0499	0.022	-2.243	0.025	-0.094	-0.006
57	-0.0933	0.023	-4.133	0.000	-0.138	-0.049
58	-0.0453	0.023	-1.988	0.047	-0.090	-0.001
60	-0.1000	0.023	-4.315	0.000	-0.145	-0.055
63	-0.0749	0.024	-3.107	0.002	-0.122	-0.028
67	-0.0900	0.025	-3.576	0.000	-0.139	-0.041
68	-0.0552	0.026	-2.135	0.033	-0.106	-0.004
73	0.1278	0.027	4.655	0.000	0.074	0.182
75	0.0594	0.028	2.139	0.033	0.005	0.114
76	0.0787	0.028	2.818	0.005	0.024	0.133
78	-0.0721	0.029	-2.520	0.012	-0.128	-0.016
79	0.1160	0.029	3.995	0.000	0.059	0.173
81	-0.0724	0.030	-2.429	0.015	-0.131	-0.014
83	-0.1022	0.030	-3.361	0.001	-0.162	-0.043
84	0.1297	0.031	4.231	0.000	0.070	0.190
85	0.0683	0.031	2.198	0.028	0.007	0.129
96	0.0970	0.035	2.747	0.006	0.028	0.166
100	0.0879	0.039	2.230	0.026	0.011	0.165
104	0.0969	0.043	2.274	0.023	0.013	0.180
117	-0.1254	0.057	-2.186	0.029	-0.238	-0.013
120	-0.1484	0.062	-2.386	0.017	-0.270	-0.026
123	-0.2042	0.068	-2.997	0.003	-0.338	-0.071
148	-0.3735	0.148	-2.522	0.012	-0.664	-0.083
153	-0.5255	0.187	-2.808	0.005	-0.893	-0.158
157	0.6875	0.240	2.861	0.004	0.216	1.159
187	8.1794	2.744	2.981	0.003	2.798	13.561
=====						
Omnibus:		264.747	Durbin-Watson:			1.995
Prob(Omnibus):		0.000	Jarque-Bera (JB):			681.229
Skew:		0.798	Prob(JB):			1.18e-148
Kurtosis:		5.549	Cond. No.			557.
=====						

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

This model has 122 different significant predictors. This model may be very complicated to interpret and implement due to the high number of significant features.

```
In [260... # Evaluate Fit

# Load Libraries
import numpy as np
from sklearn.metrics import r2_score, mean_squared_error

# Predictions
train_predictions = model.predict(features_train_pca)
test_predictions = model.predict(features_test_final)

# R2 Calculation
r2_train = r2_score(target_train, train_predictions)
r2_test = r2_score(target_test, test_predictions)

# RMSE Calculation
rmse_train = np.sqrt(mean_squared_error(target_train, train_predictions))
rmse_test = np.sqrt(mean_squared_error(target_test, test_predictions))

# Output the results
print(" P-Value Threshold 0.05")
print(f"Training R2: {r2_train}, Training RMSE: {rmse_train}")
print(f"Test R2: {r2_test}, Test RMSE: {rmse_test}")

P-Value Threshold 0.05
Training R2: 0.46316813334944595, Training RMSE: 0.8662601376571032
Test R2: 0.3515004447159483, Test RMSE: 0.9432834155315127
```

In []:

Can I train a Neural network model to predict total war

```
In [261... # Create a train test split.
# Import packages.
import pandas as pd
from sklearn.model_selection import train_test_split
# create features
# Drop target variables and unnecessary columns
features = model_df.drop(columns=['name', 'Total WAR', 'next_season_total_war'])
target = model_df['next_season_total_war']
# Split into 80 % training 20% test # use train_test_split
features_train, features_test, target_train, target_test = train_test_split(features,
target, test_size=0.2, random_state=42)

# Ensure all features are numeric for modeling
features_train = features_train.apply(pd.to_numeric, errors='coerce')
features_test = features_test.apply(pd.to_numeric, errors='coerce')
target_train = pd.to_numeric(target_train, errors='coerce')
target_test = pd.to_numeric(target_test, errors='coerce')

In [262... from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score, mean_squared_error

# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(features_train)
X_test_scaled = scaler.transform(features_test)

# Build and train model
mlp = MLPRegressor(hidden_layer_sizes=(64, 32), activation='relu', max_iter=500, random_state=42)
mlp.fit(X_train_scaled, target_train)

# Predict
nn_predictions = mlp.predict(X_test_scaled)

# Evaluate Train Set
print("MLPRegressor Train R2:", r2_score(target_train, mlp.predict(X_train_scaled)))
print("MLPRegressor Train RMSE:", np.sqrt(mean_squared_error(target_train, mlp.predict(X_train_scaled))))

# Evaluate Test Set
print("MLPRegressor R2:", r2_score(target_test, nn_predictions))
print("MLPRegressor RMSE:", np.sqrt(mean_squared_error(target_test, nn_predictions)))

MLPRegressor Train R2: 0.9921080042551181
MLPRegressor Train RMSE: 0.10503223508534587
MLPRegressor R2: 0.02741397003273738
MLPRegressor RMSE: 1.1551845483454044
```

Serious signs of overfitting on this neural network model. Explains almost no variance in the data of the test set.

Build a random forest

```

In [263... # Create a train test split.
# Import packages.
import pandas as pd
from sklearn.model_selection import train_test_split
# create features
# Drop target variables and unnecessary columns
features = model_df.drop(columns=['name', 'Total WAR', 'next_season_total_war'])
target = model_df['next_season_total_war']
# Split into 80 % training 20% test # use train_test_split
features_train, features_test, target_train, target_test = train_test_split(features,
target, test_size=0.2, random_state=42)

# Ensure all features are numeric for modeling
features_train = features_train.apply(pd.to_numeric, errors='coerce')
features_test = features_test.apply(pd.to_numeric, errors='coerce')
target_train = pd.to_numeric(target_train, errors='coerce')
target_test = pd.to_numeric(target_test, errors='coerce')

In [264... # Load libraries
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor, BaggingRegressor, AdaBoostRegressor, GradientBoostingRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.preprocessing import MinMaxScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV
import warnings

# Suppress future warnings
warnings.filterwarnings("ignore", category=FutureWarning)

# Initialize the Min-Max scaler
mmscaler = MinMaxScaler()

# Define pipeline
pipe = Pipeline([("MinMaxScaler", mmscaler), ("regressor", RandomForestRegressor())])

# Create dictionary with candidate learning algorithms and their hyperparameters
search_space = [
    # Linear Regression (no hyperparameters for grid search)
    {"regressor": [LinearRegression()]},
    # Random Forest Param
    {"regressor": [RandomForestRegressor()],
     "regressor__n_estimators": [10, 100, 1000],
     "regressor__max_features": [1, 2, 3]},
    # Decision Tree Param
    {"regressor": [DecisionTreeRegressor()],
     "regressor__max_depth": [None, 2, 4, 6, 8, 10, 12, 20],
     "regressor__min_samples_split": [2, 5, 10],
     "regressor__min_samples_leaf": [1, 2, 4],
     "regressor__criterion": ['squared_error', 'friedman_mse']},
    # Bagging param
    {"regressor": [BaggingRegressor(estimator=DecisionTreeRegressor(), random_state=42)],
     "regressor__n_estimators": [10, 50, 100]},
    # Boosting decision tree param
    {"regressor": [AdaBoostRegressor(estimator=DecisionTreeRegressor(max_depth=1), random_state=42)],
     "regressor__n_estimators": [50, 100, 300],
     "regressor__learning_rate": [0.01, 0.1, 1.0]},
    # Gradient Boosting param
    {"regressor": [GradientBoostingRegressor(random_state=42)],
     "regressor__n_estimators": [50, 100, 300],
     "regressor__learning_rate": [0.01, 0.1, 0.2],
     "regressor__max_depth": [3, 5, 7]},
]

# Create grid search
gridsearch = GridSearchCV(pipe, search_space, cv=5, verbose=0)

# Fit gridsearch
best_model = gridsearch.fit(features_train, target_train)

# Print Best model
print("The best model is: ", best_model.best_estimator_)

The best model is: Pipeline(steps=[('MinMaxScaler', MinMaxScaler()),
    ('regressor',
     GradientBoostingRegressor(learning_rate=0.01, n_estimators=300,
                                random_state=42))])

```

In []:

```
In [265... #Find the R2 and RMSE for the training and test sets
from sklearn.metrics import r2_score, mean_squared_error
import numpy as np
# Predictions
train_predictions = best_model.predict(features_train)
test_predictions = best_model.predict(features_test)
# Evaluate Train Set
print("Best Estimator Train R²:", r2_score(target_train, train_predictions))
print("Best Estimator Train RMSE:", np.sqrt(mean_squared_error(target_train, train_predictions)))
# Evaluate Test Set
print("Best Estimator Test R²:", r2_score(target_test, test_predictions))
print("Best Estimator Test RMSE:", np.sqrt(mean_squared_error(target_test, test_predictions)))
```

```
Best Estimator Train R²: 0.5554431361687824
Best Estimator Train RMSE: 0.7883023952485619
Best Estimator Test R²: 0.3460816575047302
Best Estimator Test RMSE: 0.9472162006588407
```

```
In [266... #Find the best estimator
best_estimator = best_model.best_estimator_

# Get the feature names from df
feature_names = features_train.columns

# Find the importances from the gradient boost classifier
importances = best_estimator.named_steps['regressor'].feature_importances_

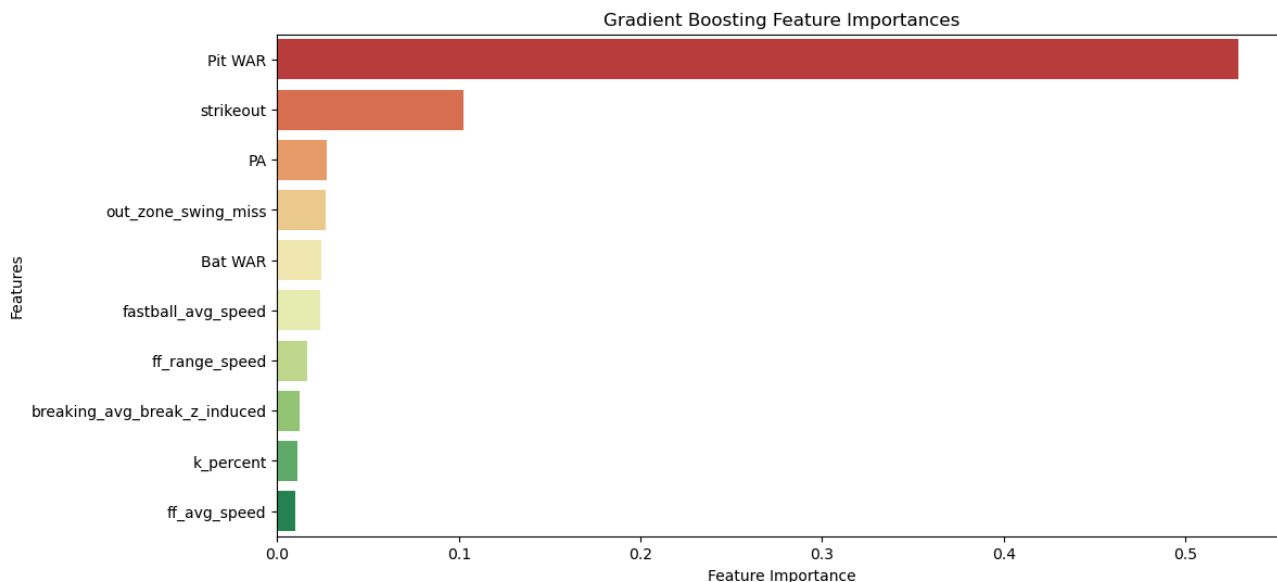
# Create a DataFrame of the importances and sort by importance
feature_importance = pd.DataFrame({'Feature': feature_names, 'Importance': importances})
# only display the top 10 features of importance.
feature_importance = feature_importance.sort_values(by='Importance', ascending=False).head(10)

# Print
print("Gradient Boosting Feature Importances (Sorted):")
print(feature_importance)
```

```
Gradient Boosting Feature Importances (Sorted):
```

	Feature	Importance
206	Pit WAR	0.529260
11	strikeout	0.102394
203	PA	0.027293
66	out_zone_swing_miss	0.026798
205	Bat WAR	0.024467
180	fastball_avg_speed	0.023720
90	ff_range_speed	0.016560
192	breaking_avg_break_z_induced	0.012045
13	k_percent	0.011401
84	ff_avg_speed	0.009827

```
In [267... # Create a bar chart of the feature importances
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
# Plot
plt.figure(figsize=(12, 6))
sns.barplot(x=feature_importance['Importance'], y=feature_importance['Feature'], palette="RdYlGn") # RdYlGn palette
plt.xlabel("Feature Importance")
plt.ylabel("Features")
plt.title("Gradient Boosting Feature Importances")
plt.show()
```

Past year pitching war is a very high predictor. This is very logical and suggests that pitchers who were good in the previous season are likely to perform similarly in the future.

```
In [268... # Apply the best model to predict 2026 WAR for players of interest.
# Use Johan Duran 2025 stats to predict 2026 WAR
# 2025 stats
jhoan_duran_2025 = filtered_df[(filtered_df['name'] == 'Jhoan Duran') & (filtered_df['year'] == 2025)]

# Mick Abel 2025 stats
mick_abel_2025 = filtered_df[(filtered_df['name'] == 'Mick Abel') & (filtered_df['year'] == 2025)]

# Griffin Jax 2025 stats
griffin_jax_2025 = filtered_df[(filtered_df['name'] == 'Griffin Jax') & (filtered_df['year'] == 2025)]

# Taj Bradley 2025 stats
taj_bradley_2025 = filtered_df[(filtered_df['name'] == 'Taj Bradley') & (filtered_df['year'] == 2025)]

# Kendry Rojas 2025 stats
kendry_rojas_2025 = filtered_df[(filtered_df['name'] == 'Kendry Rojas') & (filtered_df['year'] == 2025)]

# Predict 2026 WAR for Johan Duran
jhoan_duran_2026_war = best_model.predict(jhoan_duran_2025.drop(columns=['name', 'Total WAR', 'next_season_total_war']))
print(f"Predicted 2026 WAR for Johan Duran: {jhoan_duran_2026_war[0]}")

# Predict 2026 WAR for Mick Abel
mick_abel_2026_war = best_model.predict(mick_abel_2025.drop(columns=['name', 'Total WAR', 'next_season_total_war']))
print(f"Predicted 2026 WAR for Mick Abel: {mick_abel_2026_war[0]}")

# Predict 2026 WAR for Griffin Jax
griffin_jax_2026_war = best_model.predict(griffin_jax_2025.drop(columns=['name', 'Total WAR', 'next_season_total_war']))
print(f"Predicted 2026 WAR for Griffin Jax: {griffin_jax_2026_war[0]}")

# Predict 2026 WAR for Taj Bradley
taj_bradley_2026_war = best_model.predict(taj_bradley_2025.drop(columns=['name', 'Total WAR', 'next_season_total_war']))
print(f"Predicted 2026 WAR for Taj Bradley: {taj_bradley_2026_war[0]}")
```

Predicted 2026 WAR for Johan Duran: 1.48351654082551
 Predicted 2026 WAR for Mick Abel: 0.30546087363292845
 Predicted 2026 WAR for Griffin Jax: 1.1191770053241974
 Predicted 2026 WAR for Taj Bradley: 1.399116449980448

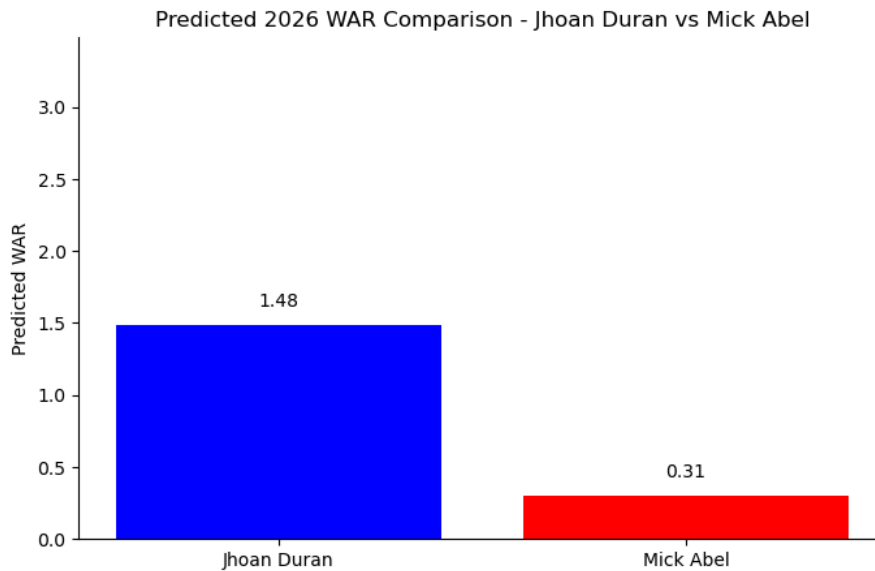
Evaluate the trades by projecting future war of all the minnesota twins pitcher for pitcher trades at the deadline.

Pitcher <-> Pitcher trades to evaluate

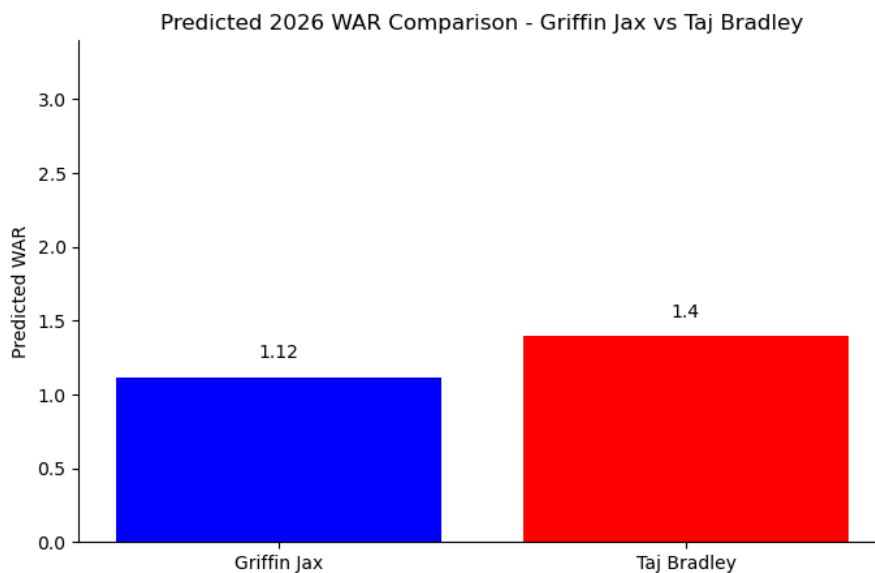
Johan Duran - Mick Abel Phillies Griffin Jax - Taj Bradley Rays Louis Varland - Kendry Rojas Blue Jays

```
In [269... # Build a bar chart to compare the two players predicted 2026 WAR
import matplotlib.pyplot as plt
import numpy as np
players = ['Jhoan Duran', 'Mick Abel']
predicted_war = [jhoan_duran_2026_war[0], mick_abel_2026_war[0]]
plt.figure(figsize=(8, 5))
bars = plt.bar(players, predicted_war, color=['blue', 'red'])
plt.title('Predicted 2026 WAR Comparison - Jhoan Duran vs Mick Abel')
plt.ylabel('Predicted WAR')
# Add value labels on top of bars
for bar in bars:
    yval = bar.get_height()
```

```
plt.text(bar.get_x() + bar.get_width()/2, yval + 0.1, round(yval, 2), ha='center', va='bottom')
plt.ylim(0, max(predicted_war) + 2)
# Remove top and right borders
import seaborn as sns
sns.despine()
plt.show()
```



```
In [270...] # Build a bar chart to compare the two players predicted 2026 WAR
import matplotlib.pyplot as plt
import numpy as np
players = ['Griffin Jax', 'Taj Bradley']
predicted_war = [griffin_jax_2026_war[0], taj_bradley_2026_war[0]]
plt.figure(figsize=(8, 5))
bars = plt.bar(players, predicted_war, color=['blue', 'red'])
plt.title('Predicted 2026 WAR Comparison - Griffin Jax vs Taj Bradley')
plt.ylabel('Predicted WAR')
# Add value labels on top of bars
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.1, round(yval, 2), ha='center', va='bottom')
plt.ylim(0, max(predicted_war) + 2)
# Remove top and right borders
import seaborn as sns
sns.despine()
plt.show()
```



```
In [271...] features_train.shape
```

```
Out[271...] (1808, 207)
```

Retrain the backwards stepwise model in order to apply it

```
In [272... # Create a train test split.
# Import packages.
import pandas as pd
from sklearn.model_selection import train_test_split
# create features
# Drop target variables and unnecessary columns
features = model_df.drop(columns=['name', 'Total WAR', 'next_season_total_war'])
target = model_df['next_season_total_war']
# Split into 80 % training 20% test # use train_test_split
features_train, features_test, target_train, target_test = train_test_split(features,
target, test_size=0.2, random_state=42)

# Ensure all features are numeric for modeling
features_train = features_train.apply(pd.to_numeric, errors='coerce')
features_test = features_test.apply(pd.to_numeric, errors='coerce')
target_train = pd.to_numeric(target_train, errors='coerce')
target_test = pd.to_numeric(target_test, errors='coerce')
```

```
In [273... # Build stepwise regression with a p value threshold of 0.05

# Load Libraries
import pandas as pd
import statsmodels.api as sm
import warnings
import numpy as np

# Suppress Warning
warnings.simplefilter("ignore", category=RuntimeWarning)

# Add a constant to the model (intercept)
features_train = sm.add_constant(features_train)

# Fit the initial model
model = sm.OLS(target_train, features_train).fit()

# Backward stepwise regression
while True:
    # Get the p-values of the model
    p_values = model.pvalues
    # Find the maximum p-value
    max_p_value = p_values.max()

    # If the maximum p-value is greater than 0.05, remove that variable
    if max_p_value > 0.05:
        # Identify the variable to remove
        variable_to_remove = p_values.idxmax()
        # Skip removing 'const'
        if variable_to_remove == 'const':
            break
        # Remove the variable from the model
        features_train = features_train.drop(variable_to_remove, axis=1)
        # Remove from test data as well
        features_test = features_test.drop(variable_to_remove, axis=1)
        # Refit the model
        model = sm.OLS(target_train, features_train).fit()
    else:
        break

# Print the summary of the final model # Commented out for length, Includes too many features
#print("Final model summary:")
#print(model.summary())

# Prepare the test set for prediction
# Add constant to test set
features_test_final = features_test.copy()
#features_test_final = sm.add_constant(features_test)
```

```
In [274... # Print all the features in the final model
print("Features in the final model:")
print(features_train.columns)
```

```
Features in the final model:
Index(['const', 'year', 'player_age', 'p_game', 'p_formatted_ip', 'pa', 'ab',
      'single', 'double', 'triple',
      ...
      'n_offspeed_formatted', 'offspeed_avg_speed', 'offspeed_avg_break_x',
      'offspeed_avg_break_z', 'offspeed_avg_break_z_induced',
      'offspeed_avg_break', 'offspeed_range_speed', 'PA', 'Bat WAR',
      'Pit WAR'],
      dtype='object', length=123)
```

Pitcher <-> Pitcher trades to evaluate

Johan Duran - Mick Abel Phillies Griffin Jax - Taj Bradley Rays Louis Varland - Kendry Rojas Blue Jays

```
In [275... # Use Johan Duran 2025 stats to predict 2026 WAR
# 2025 stats
jhoan_duran_2025 = filtered_df[(filtered_df['name'] == 'Jhoan Duran') & (filtered_df['year'] == 2025)]

# Mick Abel 2025 stats
mick_abel_2025 = filtered_df[(filtered_df['name'] == 'Mick Abel') & (filtered_df['year'] == 2025)]

# Griffin Jax 2025 stats
griffin_jax_2025 = filtered_df[(filtered_df['name'] == 'Griffin Jax') & (filtered_df['year'] == 2025)]

In [ ]:

In [276... # use the model to predict 2026 WAR for both players

# Features set for both players
jhoan_duran_2025_features = jhoan_duran_2025.drop(columns=['name', 'Total WAR', 'next_season_total_war'])
mick_abel_2025_features = mick_abel_2025.drop(columns=['name', 'Total WAR', 'next_season_total_war'])

# Add constant and reindex to match training columns
jhoan_duran_2025_features = sm.add_constant(jhoan_duran_2025_features, has_constant='add')
mick_abel_2025_features = sm.add_constant(mick_abel_2025_features, has_constant='add')

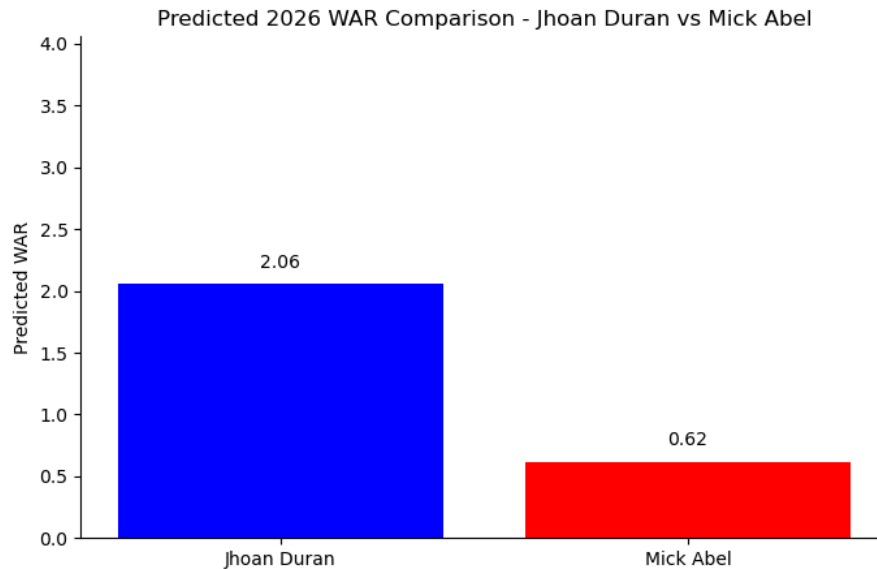
jhoan_duran_2025_features = jhoan_duran_2025_features.reindex(columns=features_train.columns, fill_value=0)
mick_abel_2025_features = mick_abel_2025_features.reindex(columns=features_train.columns, fill_value=0)

# Predict
jhoan_duran_2026_war = model.predict(jhoan_duran_2025_features)
mick_abel_2026_war = model.predict(mick_abel_2025_features)

print(f"Predicted 2026 WAR for Johan Duran: {jhoan_duran_2026_war.values[0]}")
print(f"Predicted 2026 WAR for Mick Abel: {mick_abel_2026_war.values[0]}")

Predicted 2026 WAR for Johan Duran: 2.0578804648246867
Predicted 2026 WAR for Mick Abel: 0.6178779426690726
```

```
In [277... # Build a bar chart to compare the two players predicted 2026 WAR
import matplotlib.pyplot as plt
import numpy as np
players = ['Jhoan Duran', 'Mick Abel']
predicted_war = [jhoan_duran_2026_war.values[0], mick_abel_2026_war.values[0]]
plt.figure(figsize=(8, 5))
bars = plt.bar(players, predicted_war, color=['blue', 'red'])
plt.title('Predicted 2026 WAR Comparison - Jhoan Duran vs Mick Abel')
plt.ylabel('Predicted WAR')
# Add value labels on top of bars
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.1, round(yval, 2), ha='center', va='bottom')
plt.ylim(0, max(predicted_war) + 2)
# Remove top and right borders
import seaborn as sns
sns.despine()
plt.show()
```



Repeat the process Griffin Jax - Taj Bradley Rays

```
In [278... # 2025 stats Griffin Jax
griffin_jax_2025 = filtered_df[(filtered_df['name'] == 'Griffin Jax') & (filtered_df['year'] == 2025)]

# Taj Bradley 2025 stats
# 2025 stats
taj_bradley_2025 = filtered_df[(filtered_df['name'] == 'Taj Bradley') & (filtered_df['year'] == 2025)]

In [279... # use the model to predict 2026 WAR for both players

# Features set for both players
griffin_jax_2025_features = griffin_jax_2025.drop(columns=['name', 'Total WAR', 'next_season_total_war'])
taj_bradley_2025_features = taj_bradley_2025.drop(columns=['name', 'Total WAR', 'next_season_total_war'])

# Add constant and reindex to match training columns
griffin_jax_2025_features = sm.add_constant(griffin_jax_2025_features, has_constant='add')
taj_bradley_2025_features = sm.add_constant(taj_bradley_2025_features, has_constant='add')

griffin_jax_2025_features = griffin_jax_2025_features.reindex(columns=features_train.columns, fill_value=0)
taj_bradley_2025_features = taj_bradley_2025_features.reindex(columns=features_train.columns, fill_value=0)

# Predict
griffin_jax_2026_war = model.predict(griffin_jax_2025_features)
taj_bradley_2026_war = model.predict(taj_bradley_2025_features)

print(f"Predicted 2026 WAR for Griffin Jax: {griffin_jax_2026_war.values[0]}")
print(f"Predicted 2026 WAR for Taj Bradley: {taj_bradley_2026_war.values[0]}")
```

Predicted 2026 WAR for Griffin Jax: 1.535976354894696
 Predicted 2026 WAR for Taj Bradley: 1.1756691432088504

```
In [280... # Build a bar chart to compare the two players predicted 2026 WAR
import matplotlib.pyplot as plt
import numpy as np
players = ['Griffin Jax', 'Taj Bradley']
predicted_war = [griffin_jax_2026_war.values[0], taj_bradley_2026_war.values[0]]
plt.figure(figsize=(8, 5))
bars = plt.bar(players, predicted_war, color=['blue', 'red'])
plt.title('Predicted 2026 WAR Comparison - Griffin Jax vs Taj Bradley')
plt.ylabel('Predicted WAR')
# Add value labels on top of bars
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.1, round(yval, 2), ha='center', va='bottom')
plt.ylim(0, max(predicted_war) + 2)
# Remove top and right borders
import seaborn as sns
sns.despine()
plt.show()
```

